DevOps

Dr. John-Paul Ore

“There are only two hard things in Computer Science: cache invalidation and naming things.”

-- Phil Karlton

Housekeeping

- Workshop 5 is due Tuesday 11:59pm AOE.
- About the Midterm (Oct 8th)…Paper, Conceptual, “Top 10 Adages”

Today’s Learning Objectives

- Familiar with types and techniques for “Retrospectives.”
- Understand “Change Ownership” in a DevOps context.

Continuous-Deployment Glossary #2

Change Ownership	A practice in which developers are responsible for software changes for all phases, including development, testing, deployment, and fixing problems.
End-user communication	A practice enabling communication with users to receive feedback and gather requirements.
Retrospectives	A practice in which team members discuss the causes and consequences of an unexpected operation outage or deployment failure.
Telemetry	A practice in which code is instrumented to compute metrics about feature use and software performance and stability.
Microservices	An architectural style in which services are created as small and often stateless instances and connected through a central discovery service and property store.

Parnin, Chris, Eric Helms, Chris Atlee, Harley Boughton, Mark Ghattas, Andy Glover, James Holman et al. "The top 10 adages in continuous deployment." IEEE Software 34, no. 3 (2017): 86-95.

You Are the Support Person

("Top 10 Adages")

Information and Software Technology 114 (2019) 217–230

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof

DevOps in practice: A multiple case study of five companies

Lucy Ellen Lwakatare[a,*], Terhi Kilamo[b], Teemu Karvonen[a], Tanja Sauvola[a], Ville Heikkilä[c], Juha Itkonen[c], Pasi Kuvaja[a], Tommi Mikkonen[d], Markku Oivo[a], Casper Lassenius[c]

[a] M3S, Faculty of Information Technology and Electrical Engineering, University of Oulu, Oulu, Finland
[b] Faculty of Information Technology and Communication Sciences, Tampere University, Tampere, Finland
[c] Department of Computer Science, Aalto University, Helsinki, Finland
[d] Department of Computer Science, University of Helsinki, Helsinki, Finland

ARTICLE INFO

Keywords:
DevOps
Continuous deployment
Agile
Operations
Development

ABSTRACT

Context: DevOps is considered important in the ability to frequently and reliably update a system in operational state. DevOps presumes cross-functional collaboration and automation between software development and operations. DevOps adoption and implementation in companies is non-trivial due to required changes in technical, organisational and cultural aspects.

Objectives: This exploratory study presents detailed descriptions of how DevOps is implemented in practice. The context of our empirical investigation is web application and service development in small and medium sized companies.

Method: A multiple-case study was conducted in five different development contexts with successful DevOps implementations since its benefits, such as quick releases and minimum deployment errors, were achieved. Data

Traditionally (and sometime today) **Dev** and **Ops** are *"Siloed"*

"I knew this organization was full of siloes.
I just didn't think they'd be so heavily defended!"

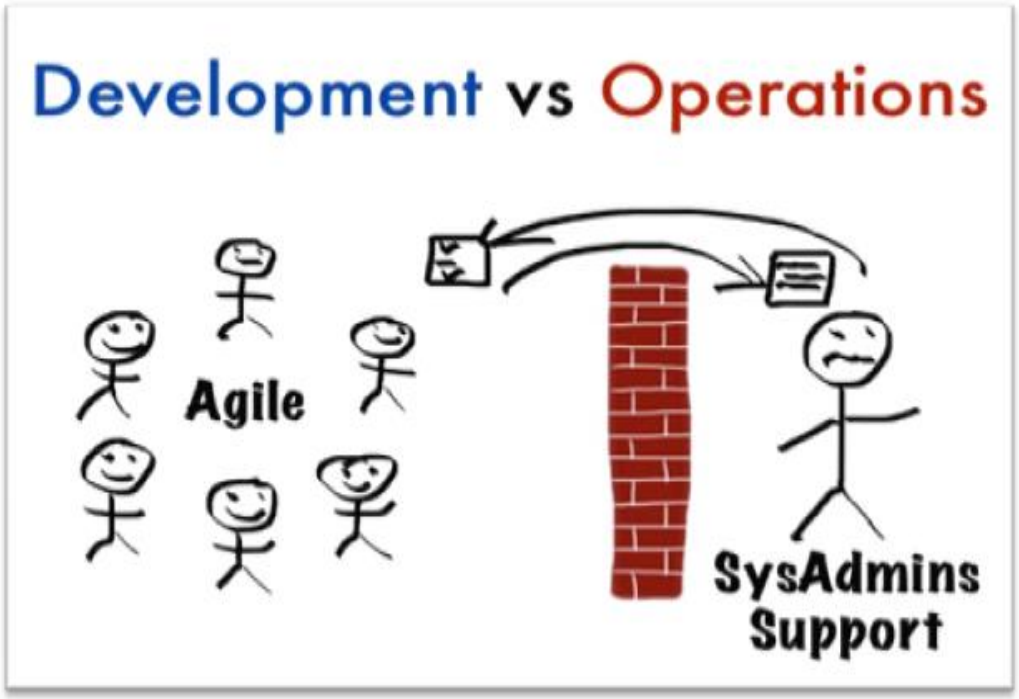
DevOps Seeks to Unite Silos

*"The interface between them in the pre-DevOps era was usually **a ticket system**: Development teams demanded the deployment of new software versions, and the operations staff manually managed those tickets.*

*Theoretically, this structure provides higher stability for software in production. However, in practice, it also results in **long delays between code updates and deployment**, as well as ineffective problem-solving processes, in which **organizational silos blame each other** for production problems."*

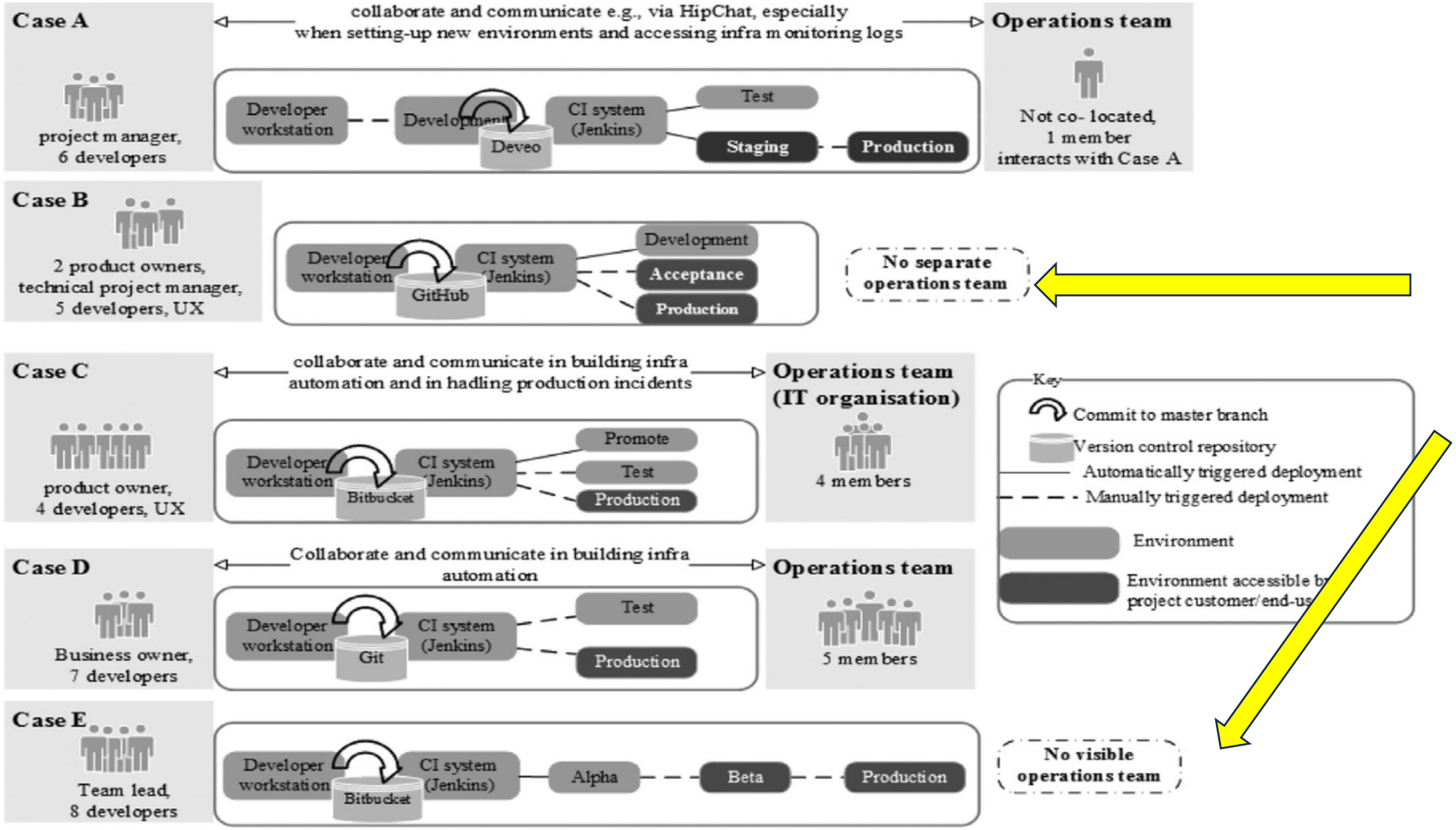
Leite, Leonardo, Carla Rocha, Fabio Kon, Dejan Milojicic, and Paulo Meirelles. "A survey of DevOps concepts and challenges." *ACM Computing Surveys (CSUR)* 52, no. 6 (2019): 1-35. https://dl.acm.org/doi/10.1145/3359981

“Wall of Confusion”



Software goes over the wall
Ops owns provisioning & rollout.

DevOps Case study of 5 Finnish companies



Dev & Ops are the same team

https://www.sciencedirect.com/science/article/pii/S0950584917302793

DevOps means Ownership and Responsibility

‘It means giving the team more responsibility and control… that you don’t handover to operations your deployment. It’s gaining confident in your deployment procedure and your ways of working’ (Developer, Case E)

‘DevOps is when you have full responsibility of the software you are developing in production ’ (PO of operations team, Case D)

‘DevOps in my opinion is that, persons who are developing the applications are also the same ones to maintain them in production. If something goes wrong then the same persons are investigating the problems and fixing them’ (IT department-Ops team lead, Case C)

https://www.sciencedirect.com/science/article/pii/S0950584917302793

“Change Ownership” (2015)

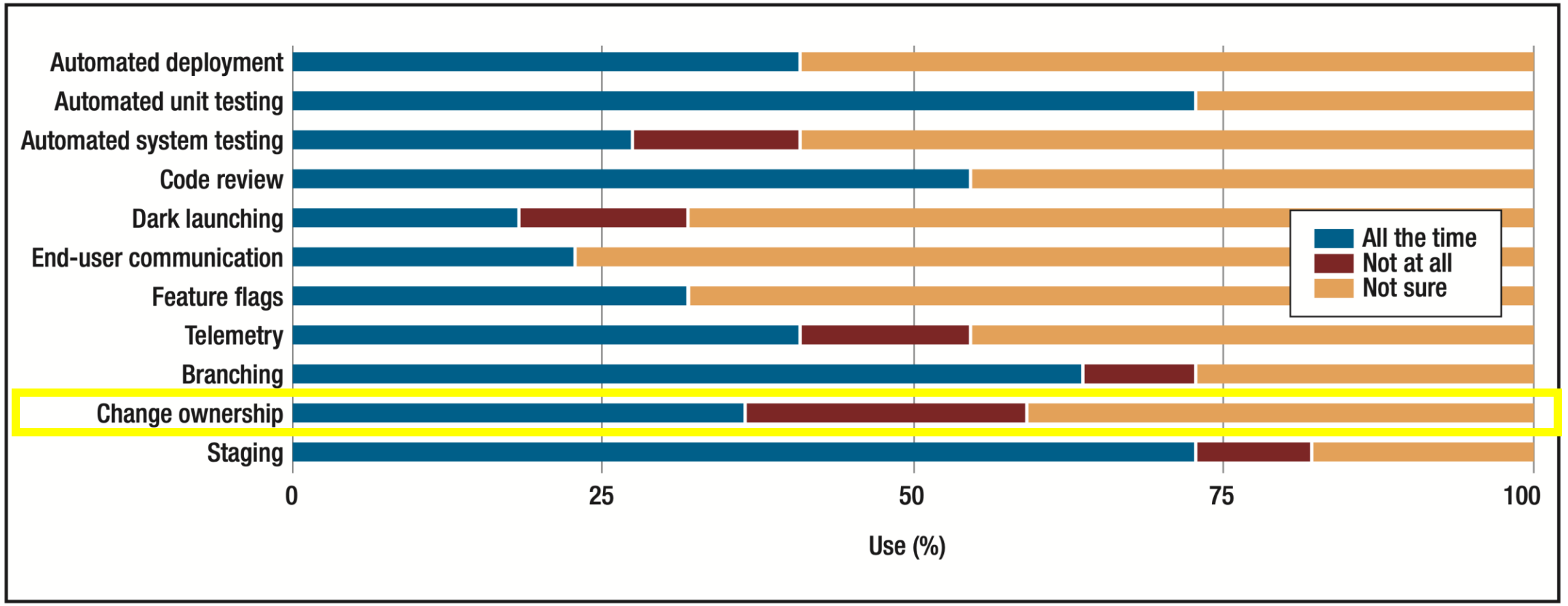


FIGURE 1. The survey respondents’ use of 11 continuous-deployment practices (17 teams from nine companies responded). For explanations of some of these practices, see Table 1.

Activity

Think-pair-share

Question: Why were (or are) *Development* and *Operations* **separated** in the first place**?**

Looking Back to Move Forward

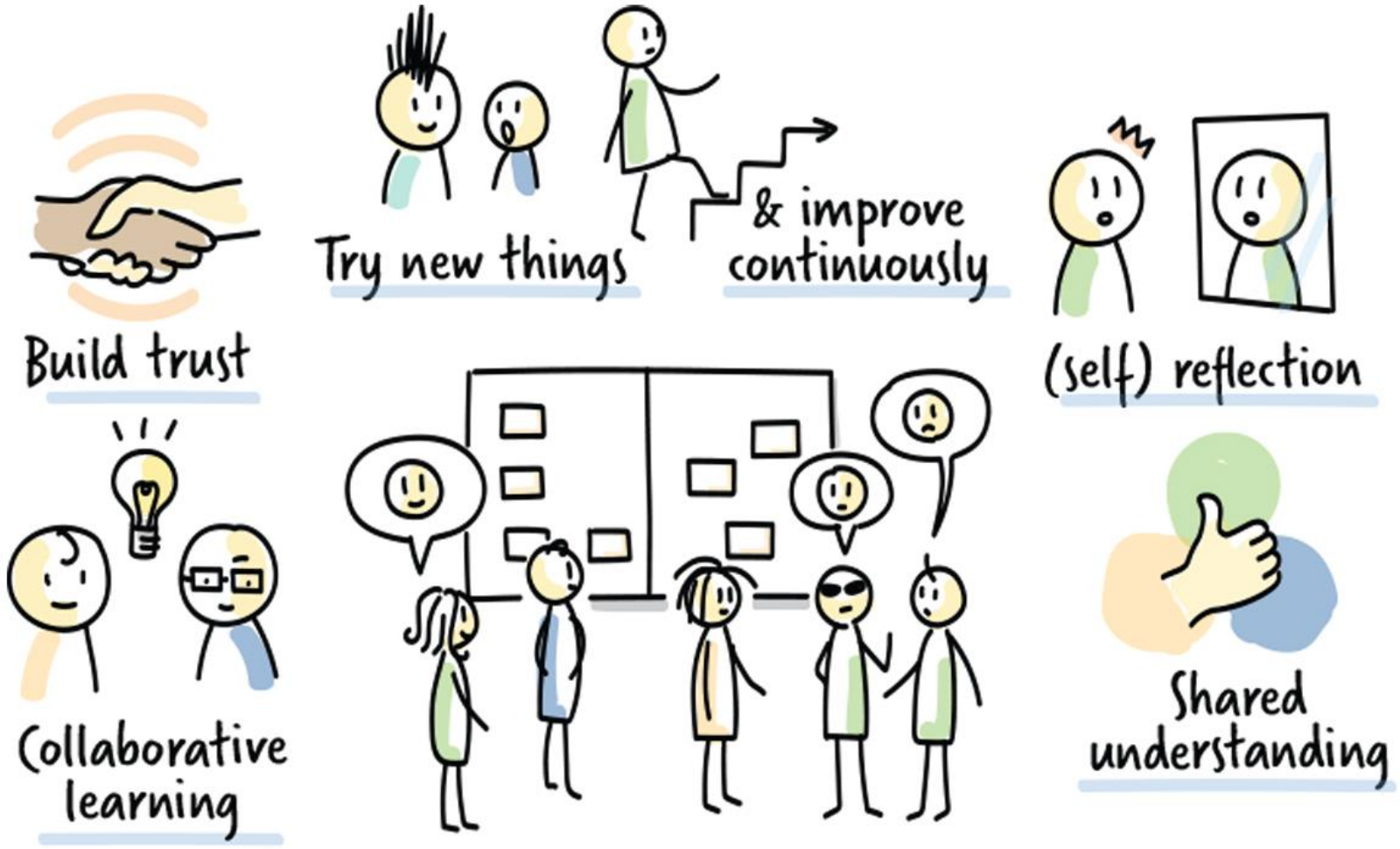
("Top 10 Adages")

Continuous-Deployment Glossary #2

Change Ownership	A practice in which developers are responsible for software changes for all phases, including development, testing, deployment, and fixing problems.
End-user communication	A practice enabling communication with users to receive feedback and gather requirements
Retrospectives	A practice in which team members discuss the causes and consequences of an unexpected operation outage or deployment failure.
Telemetry	A practice in which code is instrumented to compute metrics about feature use and software performance and stability.
Microservices	An architectural style in which services are created as small and often stateless instances and connected through a central discovery service and property store.

Parnin, Chris, Eric Helms, Chris Atlee, Harley Boughton, Mark Ghattas, Andy Glover, James Holman et al. "The top 10 adages in continuous deployment." IEEE Software 34, no. 3 (2017): 86-95.

Why Retrospectives?



Retrospectives Tips from Agile

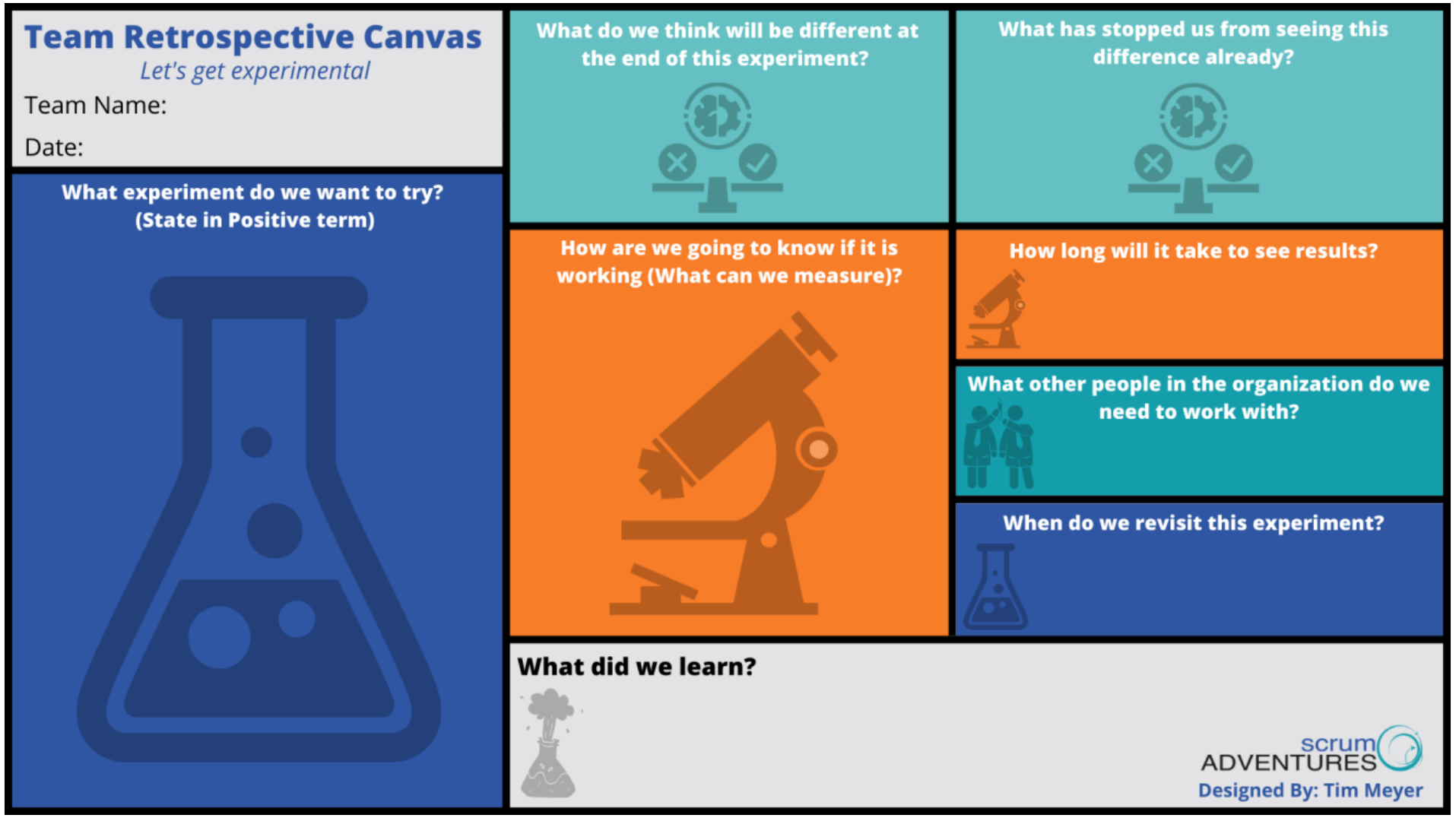
- Vocabulary.
- Ice breakers.
- Foster team psychological safety.
- Make it fun.

https://softwaredevtools.com/

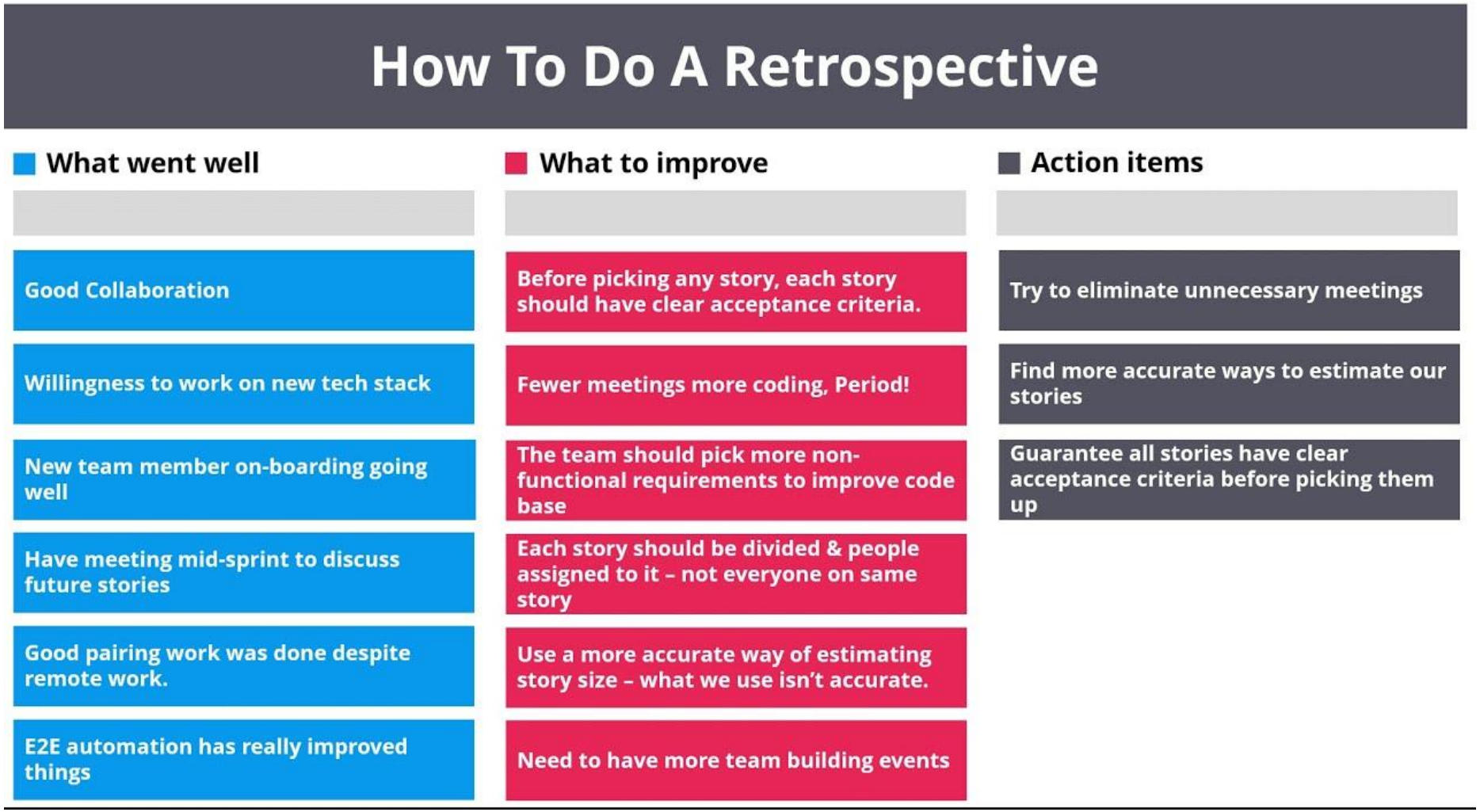
Retrospective with Theme: Outdoor Trip

Retrospective with Theme: Frog Pond

Retrospective with Theme: Experiment



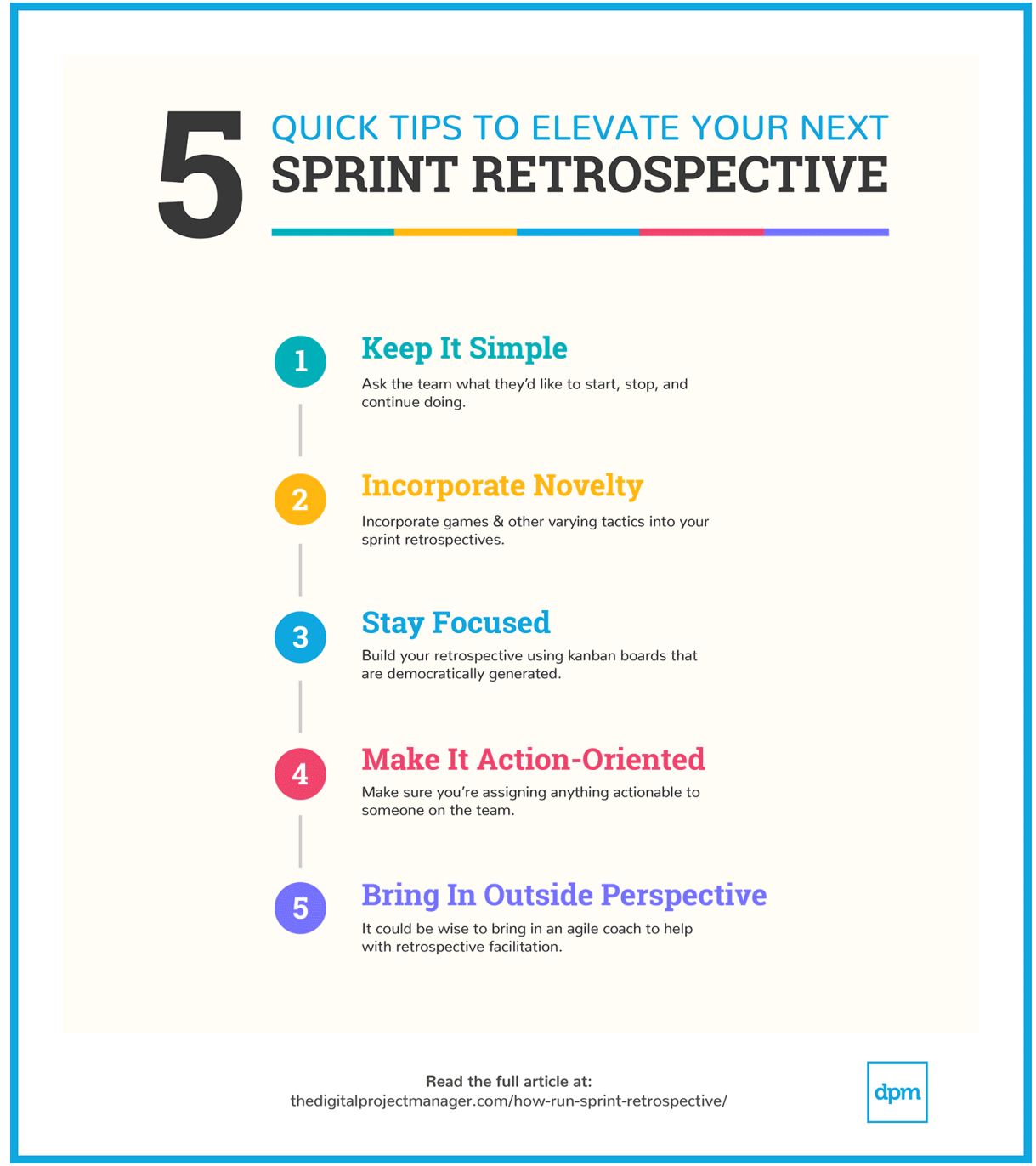
Retrospectives for Agile Sprints



https://careerprep.me/how-to-do-a-retrospective-step-by-step-playbook-and-example/

Retrospectives: Main 3 Objectives

More tips…


5
QUICK TIPS TO ELEVATE YOUR NEXT
SPRINT RETROSPECTIVE
1
Keep It Simple
Ask the team what they'd like to start, stop, and continue doing.
2
Incorporate Novelty
Incorporate games & other varying tactics into your sprint retrospectives.
3
Stay Focused
Build your retrospective using kanban boards that are democratically generated.
4
Make It Action-Oriented
Make sure you're assigning anything actionable to someone on the team.
5
Bring In Outside Perspective
It could be wise to bring in an agile coach to help with retrospective facilitation.
Read the full article at:
thedigitalprojectmanager.com/how-run-sprint-retrospective/
dpm

Tools for Retrospectives

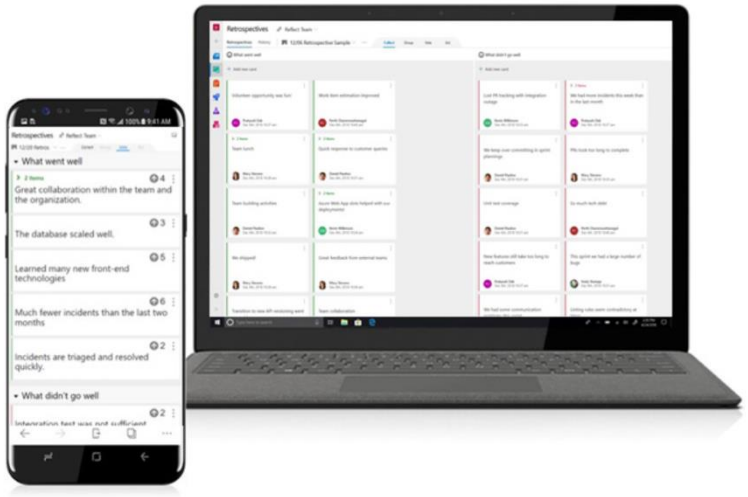
Microsoft | Garage Home About Workbench Profiles More All Microsoft

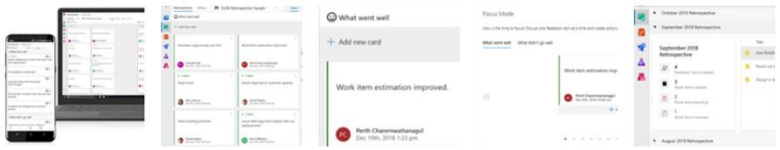
Team Retrospectives

an Azure DevOps extension

Available on

Provide feedback >





Don't just ship your products — LEARN from them. Looking back is an essential part of moving forward. Team Retrospectives, a Microsoft Garage project is an extension for Azure DevOps that provides a first-class experience for retrospectives and general feedback board scenarios. Collect feedback on your project milestones, organize and prioritize, and create and track actionable tasks to help your team improve over time. Optimized for both desktop and mobile. Celebrate the good, improve the bad, and increase productivity with Team Retrospectives.

Activity

Think-pair-share

Imagine you are promoted to manage an existing DevOps team that recently had a major service outage that caused a lot of difficult problems.

1. **Brainstorm an entertaining theme for your retrospective.**
2. **What are the top goals for your retrospective?**

Comfort the Customer with Discomfort

("Top 10 Adages")

Releases are part of the User Experience (UX)

Does the user choose?

…or not?

2017 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)

Towards Better Availability and Accountability for IoT Updates by means of a Blockchain

Aymen Boudguiga*, Nabil Bouzerna*, Louis Granboulan†, Alexis Olivereau‡,
Flavien Quesnel*, Anthony Roger* and Renaud Sirdey‡*
*IRT SystemX, 8 avenue de la Vauve 91120–Palaiseau, *firstName.lastName@irt-systemx.fr*
†Airbus Group Innovations, *louis.granboulan@airbus.com*
‡CEA–LIST, 91191 Gif-sur-Yvette, *firstName.lastName@cea.fr*

***Abstract*—Building the Internet of Things requires deploying a huge number of objects with full or limited connectivity to the Internet. Given that these objects are exposed to attackers and generally not secured-by-design, it is essential to be able to update them, to patch their vulnerabilities and to prevent hackers from enrolling them into botnets. Ideally, the update infrastructure should implement the CIA triad properties, i.e., confidentiality, integrity and availability. In this work, we investigate how the use of a blockchain infrastructure can meet these requirements, with a focus on availability. In addition, we propose a peer-to-peer mechanism, to spread updates between objects that have limited access to the Internet. Finally, we give an overview of our ongoing prototype implementation.**

***Index Terms*—Blockchain, IoT, Software updates, Availability, Accountability, Innocuousness**

I. Introduction

The Internet of Things (IoT) is today a paradigm that is changing our lives in many ways with respect to how we work, travel, entertain ourselves or communicate. Novel forms of interactions with human users, distributed intelligence, pervasiveness, new topologies, devices and communication technologies shape this evolution. From the non-expert viewpoint, the digitalization of hitherto human-centric

and *innocuousness* for IoT objects. Availability and also integrity result from the persistence property of the blockchain. That is, once an update is added to the blockchain as part of a valid block, it becomes impossible to erase it. As such, we defeat malicious entities that prevent software updates from being distributed, in order to benefit from current software vulnerabilities. That is, we ensure that updates will be always available for their intended objects. Moreover, the blockchain infrastructure may be assumed to be much more resilient to availability threats e.g., impersonations or Denial of Service (DoS), than the manufacturer's own infrastructure.

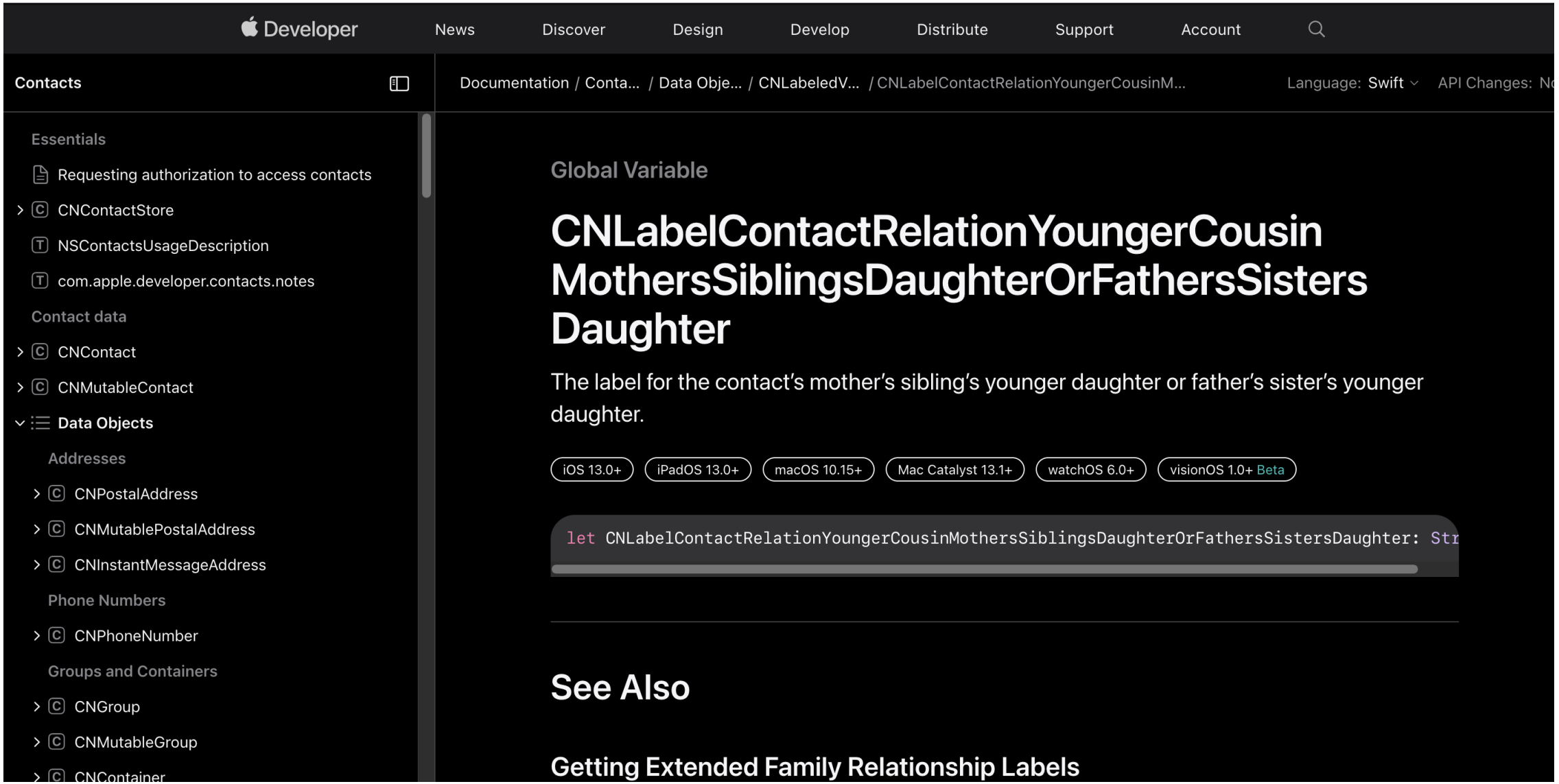
In addition, we propose to rely on some nodes in the blockchain to validate an update innocuousness before its transmission to end objects. Innocuousness checking nodes verify that a new update does not contain bugs and resists to known attacks. As such, objects will only download an update that has been approved by some innocuousness checking nodes. These nodes can belong, for example, to national cybersecurity agencies. That is, we protect legitimate objects from downloading insecure updates.

The remainder of this paper is organized as follows. Section II presents the targeted IoT architectures. Section III

Aside: Semantically Rich Variable Names 1/2

*"There are only two hard things in Computer Science: cache invalidation and **naming things**."* *-- Phil Karlton*

Aside: Semantically Rich Variable Names 2/2



https://developer.apple.com/documentation/contacts/cnlabelcontactrelationyoungercousinmotherssiblingsdaughterorfatherssistersdaughter

Summary

- The 3 main goals of Retrospectives are “What worked, what didn’t work, what can we do better.”
- “Change Ownership” in a DevOps context means that the software developer has responsibility for the consequences of changes.